Progress Report

- Increment 2-

Closet Design Game

Please use this template to describe your progress on the group project in the latest increment. Please do not change the font, font size, margins or line spacing. All the text in italic should be removed from your final submission.

1) Team Members

Ethan McConnell, edm22, ethanm177
Wesley Jordan, wsj21, Wefloo
Jessika Tyler, JessikaTyler
Chloe Patrick, cjp22n, Chloe-Patrick
Camryn Morris, cpm23c, camrynmorris1
Vanessa Campoe, vac23a, VanessaCampoe17

2) Project Title and Description

Closet Design Game is a web-based single-page fashion design application that allows users to browse clothing templates by historical era and geographic region, place garments on a mannequin, and customize those garments with interactive drawing tools. The project is being built with React, Vite, Fabric.js, Tailwind CSS, and Lucide React. The longer-term scope includes multiple mannequin/body options, outfit layering, save/load functionality in browser storage, an outfit gallery, and export/sharing of completed designs as PNG images.

3) Accomplishments and overall project status during this increment

During Increment 2, the team moved the project from an initial proof-of-concept into a working fashion studio application with state persistence:

- **Asset Normalization & Catalog Setup:** Integrated normalized vector mannequins across nine body types and added three starter garments spanning different historical eras and silhouettes. The clothing catalog schema was updated with coordinate offsets, frame compatibility, and default layering metadata to support clean asset spawning.

- **Canvas Interactivity & Layer Controls:** Connected UI controls to dynamically spawn mannequins and garments onto the Fabric.js canvas. Implemented structured layer ordering (mannequin base -> bottoms -> tops ->brush strokes) and boundary-swapping logic so users can change clothes without losing active drawing strokes.

- **Storage & Rehydration Pipeline:** Built the client-side save/load infrastructure using browser LocalStorage. Developed canvas serialization to bundle drawn paths, custom boundary metadata, and equipped garment IDs, and wired this pipeline to the Saved Outfits Gallery modal for one-click outfit reloading.

- **Automated Integration Testing:** Implemented an automated test suite using Vitest and JSDOM to validate storage and serialization of lifecycles. Achieved over 82% statement and line coverage with full test passes on all CRUD operations, corrupted JSON recovery, and boundary-condition error trapping.

Overall Project Status vs. Initial Scope

The project is currently on schedule and directly matches the core functionality proposed in our initial scope. All primary user loops—mannequin customization, garment layering, boundary-clipped texture drawing, and outfit saving/loading—are implemented and operating synchronously. With critical technical hurdles around canvas layer ordering and data persistence resolved, the application functions as a stable MVP ready for final polish, demo video recording, and catalog expansion.

4) Challenges, changes in the plan and scope of the project and things that went wrong during this increment

Challenges Encountered and Resolutions

- **Fabric.js Canvas Layering and Z-Index Management:** Managing render order dynamically between base mannequins, overlapping garments, and active drawing paths proved challenging inside the HTML5 canvas. The team resolved this by enforcing explicit defaultZIndex metadata within the catalog schema and programming deterministic re-sorting routines upon every asset spawn.
- **Preserving Brush Strokes During Boundary Swapping:** Alternating between different tops or bottoms originally risked resetting the canvas context or orphaning brush strokes outside active clipping regions. We resolved this by isolating the garment clipping path from the global drawing layer, allowing the clipping boundary to swap dynamically while retaining existing stroke vectors.
- **Mocking Browser Storage and Canvas APIs in Testing:** Simulating browser LocalStorage and Fabric.js canvas objects inside a headless Node test runner initially caused object reference and syntax rejections. We addressed this by integrating jsdom with Vitest and constructing lightweight mock canvas objects matching exact method signatures (toObject, clear, and requestRenderAll).

Changes in plan

- **Shift to Client-Side Serialization First:** Our initial architectural roadmap considered prototyping an external backend database earlier in the development cycle. We adapted the plan to focus exclusively on client-side browser LocalStorage for this increment, ensuring zero networking latency, reduced infrastructure dependencies, and faster verification of our core serialization schema.
- Rather than separate independent bounding boxes per garment piece, Pair 1 implemented a unified bounding box spanning both garments for Increment 2, scheduling independent piece clipping for Increment 3.

Things That Went Wrong

- **Rapid Storage ID Collision in Rapid Execution:** Early unit testing revealed that using millisecond-based timestamps for outfit record generation caused ID collisions when saving multiple looks in rapid succession, resulting in accidental record overwrites during batch deletions. This was corrected by incorporating asynchronous execution padding and unique string key generation.
- **Canvas Serialization Payload Validation Mismatches:** Discrepancies between how serialization methods exported object arrays versus how deserializers expected JSON strings caused test assertions to fail on malformed payloads. We rectified this by adding input guard clauses and type-checking within

parseCanvasPayload to safely trap syntax errors and missing object structures before they reached the rendering pipeline.

5) Team Member Contribution for this increment

- Vanessa Campoe (Pair 3)
  - Primary author of the progress report did not contribute to IT, RD, or Video portions. Source code contributions include outfitStorage.js, structured clothingCatalog.json started metadata, normalized started SVG assets, and authored 6-test automated integration test suite in src/utils/**tests**/sorageIntegration.test.js
- Welsey Jordan (Pair 3)
  - Co-author of RD and IT document and contributed to a section of the video. Source code contributions consisted of co-development of canvasSerialization.js and outfitStorage.js, implementing serialization parsing and LocalStorage CRUD functions.

- Jessika Tyler(Pair 1)
  - In charge of the video clipping and editing, along with source code contributions of rendering for mannequins and garments on the canvas. She also helps with fine tuning of the coordinate positioning for shirts and pants, and confining the unified garment bounding box for active drawing tools.

- Ethan McConnell(Pair 1)
  - Worked on two parts of the video for this increment. Source code contributions consisted of focusing on canvas setup, layer stacking, and base rendering logic for garment asserts on the Fabric.js canvas.

- Chloe Patrick (Pair 2)
  - Co-author of the RD and IT document, wrote source code for the UI implementation for Pair 2, developed catalog drawer controls, category and era filtering logic, and clothing selection state binding in App.jsx

- Camryn Morris (Pair 2)
  - Contributed to a section of the video and implemented styling polish across the Saved Outfits modal and UI controls. She added button hover animations and made the sidebar and DesignCanvas components responsive across desktop and mobile screens. Implemented aspect-ratio-preserving canvas scaling to retain stroke dimensions, added keyboard accessibility controls for modals, resolved branch merge conflicts with main, and managed the associated GitHub PR.

6) Plans for the next increment

During Increment 3, the team will expand on the verified studio engine and persistence pipeline established in Increment 2, shifting focus toward feature completeness, expanded creative tools, and comprehensive system documentation:

- **Garment Catalog and Asset Expansion:** Scale up the available wardrobe by integrating additional historical and cultural garments across all standardized body frames, ensuring balanced representation across eras (1890s, 1950s, 1990s) and garment categories (tops, bottoms, and accessories).

- **Per-Garment Boundary Clipping**: Refine canvas boundary clipping from the unified bounding box into independent, piece-specific clipping paths, isolating individual shirts, skirts, and pants during active drawing
- **Enhanced Styling and Canvas Tools:** Implement extended customization features on the canvas, including interactive color tinting/palette pickers for garment layers, adjustable brush sizes and textures, and an undo/redo history stack to streamline user drawing workflows.
- **UI Polish and Gallery Enhancements:** Refine the user interface with improved responsive layouts, smooth drawer transitions, and enhanced Saved Outfits Gallery management—such as thumbnail previews, outfit renaming, and multi-outfit filtering by era or body archetype.
- Add per-garment boundary clipping and the exploration of saving individually painted items as standalone catalog garments.
- **Testing Expansion and Documentation Finalization:** Broaden integration and end-to-end testing coverage across the active drawing tools and UI components, while delivering the complete final iteration documentation across the Requirements Document (RD), Integration Test (IT) document, and final product demonstration.

7) Stakeholder Communication

Dear Stakeholders,

I am writing to share an update on the Closet Design Game. With our latest release, the application has transitioned from an early prototype into a fully functioning digital design studio.

Key Progress & Milestones Achieved:
• Catalog & Asset Architecture: We standardized our historical vector catalog across nine core silhouettes and added initial historical garments representing the 1890s, 1950s, and 1990s. Each asset now includes precise coordinate anchoring and layering rules.
• Layer-Aware Design Canvas: Users can dynamically swap mannequin frames and style garments in real time. We established a strict layering order—mannequin base, lower-body garments, upper-body garments, and custom surface drawing—ensuring proper visual depth.
• Boundary-Preserved Styling: Users can alternate between different silhouettes without losing their custom drawings or texture work. The application re-binds the artwork to the active garment boundary dynamically.
• Client-Side Outfit Archiving: A lightweight persistence engine allows users to save, review, and reload full outfit configurations directly through the Saved Outfits Gallery.
• Quality Verification: Automated testing for the saving and serialization pipeline achieved over 82% coverage, validating reliability against data corruption and edge cases.

Engineering Challenges & Adjustments:
As with any graphics-heavy application, coordinating multi-layered vector rendering presented challenges. Early builds risked clearing active brushwork when changing clothing layers. We resolved this by decoupling custom artwork layers from base garment templates, preserving client designs across swaps. Additionally, to ensure peak client-side performance, we prioritized browser-based storage for this phase, validating state serialization before introducing cloud synchronization.

Next Steps:
Our next release will focus on expanding the historical wardrobe catalog, introducing color-palette pickers and adjustable brush tools, and refining the user gallery interface.

Thank you for your continued support as we build out this creative platform.

Sincerely,
Vanessa Campoe
Closet Design Game Developer

8) **Link to video**
https://www.youtube.com/watch?v=DQeTyAecpdU